Using JsonCSVMaskr

JsonCSVMaskr

April 29, 2026

1 Overview

JsonCSVMaskr masks selected fields in JSON and CSV files with deterministic HMAC-SHA256 values. The same source value, field, and secret key produce the same masked value, which makes the package useful for repeatable test data creation. Each run exports masked data, a masking key, normalized CSV tables for nested JSON, and a small replication script.

2 CSV Example

The first example uses a bundled CSV file. CSV masking records a zero-based `RowIndex` in `masking_key.csv`.

```
> get_csv_fields("sample.csv")

[1] "CustomerID" "Name"       "Email"      "Phone"      "SSN"
[6] "Department"

> output_folder <- "~/Documents/MASKED/csv-example"
> unlink(output_folder, recursive = TRUE, force = TRUE)
> datamaskr(
+   input_file = "sample.csv",
+   output_folder = output_folder,
+   secret_key = "your-secret-key",
+   mask_fields = c("root.Name", "root.Email")
+ )
> list.files(output_folder)

[1] "JsonCSVMaskr-replication.R" "data.csv"
[3] "masking_key.csv"            "replicate_masking.R"
```

The masked table snippet:

```
> csv_data <- read.csv(
+   file.path(output_folder, "data.csv"),
+   check.names = FALSE
+ )
> snippet(csv_data)
```

```
  CustomerID         Name        Email    Phone         SSN
1       C001 SH6VgIskdz7o 3kKlU36X11ww 555-1234 111-22-3333
2       C002 zOyTzfFSeEWy kEczbCWBfWGq 555-5678 444-55-6666
3       C003 2/HKeIOy2/vq PQkquYbhjELR 555-9999 777-88-9999
```

The masking key snippet shows concrete row indexes:

```
> csv_key <- read.csv(
+   file.path(output_folder, "masking_key.csv"),
+   check.names = FALSE
+ )
> snippet(csv_key, rows = 6, cols = 4)

                Original       Masked Field RowIndex
1             John Smith SH6VgIskdz7o  Name        0
2  john.smith@example.com 3kKlU36X11ww Email        0
3               Jane Doe zOyTzfFSeEWy  Name        1
4    jane.doe@example.com kEczbCWBfWGq Email        1
5            Bob Johnson 2/HKeIOy2/vq  Name        2
6 bob.johnson@example.com PQkquYbhjELR Email        2
```

3 JSON Array Example

The next example uses a standard JSON array of objects. Ordinary JSON does not always have a flat source row index, so `RowIndex` may be `NA`; the masked data and field mapping are still valid.

```
> head(get_json_fields("sample.json"), 10)

[1] "root.age"   "root.email" "root.name"

> json_output_folder <- "~/Documents/MASKED/json-example"
> unlink(json_output_folder, recursive = TRUE, force = TRUE)
> datamaskr(
+   input_file = "sample.json",
+   output_folder = json_output_folder,
+   secret_key = "your-secret-key",
+   mask_fields = c("root.name", "root.email")
+ )
> list.files(json_output_folder)

[1] "JsonCSVMaskr-replication.R" "data.csv"
[3] "masking_key.csv"            "replicate_masking.R"
[5] "sample_masked.json"

> json_data <- read.csv(
+   file.path(json_output_folder, "data.csv"),
+   check.names = FALSE
+ )
> snippet(json_data)
```

```
  age        email         name  root_id
1  30 oBE6yTIOt2RS Vjg9M4nazX9B 20265275
2  25 QVj4fCtPU3YW nf6TICe8KBY+ 8f7b270e
```

```
> json_key <- read.csv(
+   file.path(json_output_folder, "masking_key.csv"),
+   check.names = FALSE
+ )
> snippet(json_key, rows = 4, cols = 4)
```

```
           Original       Masked Field RowIndex
1             Alice Vjg9M4nazX9B  name       NA
2 alice@example.com oBE6yTIOt2RS email       NA
3               Bob nf6TICe8KBY+  name       NA
4   bob@example.com QVj4fCtPU3YW email       NA
```

4 NDJSON Example

NDJSON stores one JSON object per line. JsonCSVMaskr reads this loose-record format and writes a masked `.ndjson` file plus CSV tables.

```
> head(get_json_fields("sample_ndjson.json"), 10)
```

```
[1] "root.email" "root.id"    "root.name"
```

```
> ndjson_output_folder <- "~/Documents/MASKED/ndjson-example"
> unlink(ndjson_output_folder, recursive = TRUE, force = TRUE)
> datamaskr(
+   input_file = "sample_ndjson.json",
+   output_folder = ndjson_output_folder,
+   secret_key = "your-secret-key",
+   mask_fields = c("root.email", "root.name")
+ )
> list.files(ndjson_output_folder)
```

```
[1] "JsonCSVMaskr-replication.R"  "data.csv"
[3] "masking_key.csv"             "replicate_masking.R"
[5] "sample_ndjson_masked.ndjson"
```

```
> ndjson_key <- read.csv(
+   file.path(ndjson_output_folder, "masking_key.csv"),
+   check.names = FALSE
+ )
> snippet(ndjson_key, rows = 4, cols = 4)
```

```
         Original       Masked Field RowIndex
1 one@example.com eLxDZM2HrQ4y email       NA
2             One ZS9WNehWPvGD  name       NA
3 two@example.com QyzNENvIZ7Dz email       NA
4             Two UE1ZmSg+vGq0  name       NA
```

5 Envelope JSON Example

Envelope JSON keeps records under a property such as `items`, `records`, or `data`. The package detects the record collection and masks fields inside it.

```
> head(get_json_fields("sample_envelope.json"), 10)

[1] "root.email" "root.id"

> envelope_output_folder <- "~/Documents/MASKED/envelope-example"
> unlink(envelope_output_folder, recursive = TRUE, force = TRUE)
> datamaskr(
+   input_file = "sample_envelope.json",
+   output_folder = envelope_output_folder,
+   secret_key = "your-secret-key",
+   mask_fields = c("root.email")
+ )
> list.files(envelope_output_folder)

[1] "JsonCSVMaskr-replication.R"  "data.csv"
[3] "masking_key.csv"             "replicate_masking.R"
[5] "sample_envelope_masked.json"

> envelope_key <- read.csv(
+   file.path(envelope_output_folder, "masking_key.csv"),
+   check.names = FALSE
+ )
> snippet(envelope_key, rows = 4, cols = 4)

         Original       Masked Field RowIndex
1 one@example.com eLxDZM2HrQ4y email       NA
2 two@example.com QyzNENvIZ7Dz email       NA
```

6 Header-Array JSON Example

Some exports store the first row as headers and subsequent rows as values. The header-array example below masks the `email` column.

```
> get_json_fields("sample_header_array.json")

[1] "root.email" "root.id"    "root.name"

> header_output_folder <- "~/Documents/MASKED/header-array-example"
> unlink(header_output_folder, recursive = TRUE, force = TRUE)
> datamaskr(
+   input_file = "sample_header_array.json",
+   output_folder = header_output_folder,
+   secret_key = "your-secret-key",
+   mask_fields = c("root.email")
+ )
> list.files(header_output_folder)
```

```
[1] "JsonCSVMaskr-replication.R"      "data.csv"
[3] "masking_key.csv"                 "replicate_masking.R"
[5] "sample_header_array_masked.json"
```

```
> header_key <- read.csv(
+   file.path(header_output_folder, "masking_key.csv"),
+   check.names = FALSE
+ )
> snippet(header_key, rows = 4, cols = 4)
```

```
         Original       Masked Field RowIndex
1 one@example.com eLxDZM2HrQ4y email       NA
2 two@example.com QyzNENvIZ7Dz email       NA
```

7 Complex JSON And Tidyverse Joins

Nested JSON exports are normalized into related CSV tables. This example masks organization names, owner emails, and task assignee emails, then joins the generated CSVs with tidyverse pipelines. The package creates synthetic key columns such as `root_id` and `projects_id`; use those columns to connect parent and child tables without exposing original source IDs.

```
> head(get_json_fields("sample_complex.json"), 15)
```

```
 [1] "root.org_id"
 [2] "root.org_name"
 [3] "root.owner"
 [4] "root.owner.email"
 [5] "root.owner.name"
 [6] "root.projects"
 [7] "root.projects.project_code"
 [8] "root.projects.project_name"
 [9] "root.projects.tasks"
[10] "root.projects.tasks.assignee_email"
[11] "root.projects.tasks.status"
[12] "root.projects.tasks.task_id"
```

```
> complex_output_folder <- "~/Documents/MASKED/complex-example"
> unlink(complex_output_folder, recursive = TRUE, force = TRUE)
> datamaskr(
+   input_file = "sample_complex.json",
+   output_folder = complex_output_folder,
+   secret_key = "your-secret-key",
+   mask_fields = c(
+     "root.org_name",
+     "root.owner.email",
+     "root.projects.tasks.assignee_email"
+   )
+ )
> list.files(complex_output_folder)
```

```
[1] "JsonCSVMaskr-replication.R" "data.csv"
[3] "masking_key.csv"            "owner.csv"
[5] "projects.csv"               "projects_tasks.csv"
[7] "replicate_masking.R"        "sample_complex_masked.json"
```

Only snippets are shown below. First, read the generated CSV tables:

```
> orgs <- readr::read_csv(
+   file.path(complex_output_folder, "data.csv"),
+   show_col_types = FALSE
+ )
> owners <- readr::read_csv(
+   file.path(complex_output_folder, "owner.csv"),
+   show_col_types = FALSE
+ )
> projects <- readr::read_csv(
+   file.path(complex_output_folder, "projects.csv"),
+   show_col_types = FALSE
+ )
> tasks <- readr::read_csv(
+   file.path(complex_output_folder, "projects_tasks.csv"),
+   show_col_types = FALSE
+ )
> list(
+   orgs = names(orgs),
+   owners = names(owners),
+   projects = names(projects),
+   tasks = names(tasks)
+ )

$orgs
[1] "org_id"   "org_name" "root_id"

$owners
[1] "email"    "name"     "owner_id" "root_id"

$projects
[1] "project_code" "project_name" "projects_id"  "root_id"

$tasks
[1] "assignee_email" "projects_id"    "root_id"        "status"
[5] "task_id"        "tasks_id"
```

Join organization, owner, project, and task rows:

```
> joined <- orgs |>
+   dplyr::select(root_id, org_id, org_name) |>
+   dplyr::left_join(
+     owners |> dplyr::select(root_id, owner_email = email),
```

```
+     by = "root_id"
+   ) |>
+   dplyr::left_join(
+     projects |> dplyr::select(root_id, projects_id, project_code, project_name),
+     by = "root_id"
+   ) |>
+   dplyr::left_join(
+     tasks |> dplyr::select(projects_id, task_id, assignee_email, status),
+     by = "projects_id"
+   )
> snippet(joined, rows = 6, cols = 8)

   root_id  org_id     org_name  owner_email projects_id project_code
1 20265275 ORG-001 R1u+/udR0JUO X2z4EX5nBvM3    71e9834a        P-100
2 20265275 ORG-001 R1u+/udR0JUO X2z4EX5nBvM3    71e9834a        P-100
3 20265275 ORG-001 R1u+/udR0JUO X2z4EX5nBvM3    d59c9208        P-200
4 8f7b270e ORG-002 BXanndUPxsLm OjovOcR3+7pe    6d39404c        P-300
     project_name task_id
1 Customer Portal     T-1
2 Customer Portal     T-2
3    Data Cleanup     T-3
4  Records Review     T-4
```

The masking key confirms which original values were masked:

```
> complex_key <- read.csv(
+   file.path(complex_output_folder, "masking_key.csv"),
+   check.names = FALSE
+ )
> snippet(complex_key, rows = 8, cols = 4)

                Original       Masked                         Field
1           Northwind Lab R1u+/udR0JUO                      org_name
2 avery.stone@example.com X2z4EX5nBvM3                    owner.email
3     dev.one@example.com zUBqV6LkzVKt projects.tasks.assignee_email
4     dev.two@example.com t5CFaDRGlJnV projects.tasks.assignee_email
5     analyst@example.com /ekWHckoKMe6 projects.tasks.assignee_email
6          Contoso Health BXanndUPxsLm                      org_name
7  morgan.lee@example.com OjovOcR3+7pe                    owner.email
8    reviewer@example.com yUrN4FoWgozf projects.tasks.assignee_email
  RowIndex
1       NA
2       NA
3       NA
4       NA
5       NA
6       NA
7       NA
8       NA
```

8 Replication Script

Each run writes `replicate_masking.R`. It loads JsonCSVMaskr, remembers the original input and output paths, and can be opened in R and run directly. The snippet below shows only the path-setting lines from the CSV example.

```
> replication <- readLines(
+   file.path(output_folder, "replicate_masking.R"),
+   warn = FALSE
+ )
> cat(
+   "First few lines of replicate_masking.R:\n\n",
+   paste(head(replication, 5), collapse = "\n"),
+   "\n"
+ )

First few lines of replicate_masking.R:

 # Replicate a JsonCSVMaskr masking run.
# Open this file in R and run it to repeat the same masking operation.

script_path <- tryCatch(normalizePath(sys.frame(1)$ofile, winslash = '/', mustWork = FALSE), err
script_dir <- if (!is.null(script_path) && nzchar(script_path)) dirname(script_path) else getwd(
```

9 Launching The GUI

Users who prefer a browser-based interface can launch the Shiny GUI from R:

```
library(JsonCSVMaskr)
run_datamaskr()
```

The GUI defaults output to `~/Documents/MASKED`. On a local Windows, macOS, or Linux desktop session, Browse buttons use real filesystem paths so replication scripts do not store Shiny temporary upload paths.